

Cahier des charges - DiaspoConnect

Cahier des charges — DiaspoConnect

Auteur : Fatoumata Diallo **Projet :** Application mobile Flutter — Semaine 5/6

1. Contexte et problématique

Les membres d'une même communauté de diaspora (originaires d'un même pays ou d'une même région) sont souvent dispersés dans plusieurs villes et pays, sans outil simple pour se repérer entre eux, s'informer des événements communautaires ou s'entraider au quotidien (logement, cours, services). DiaspoConnect répond à ce besoin par une application mobile centrée sur le lien social et l'entraide de proximité.

2. Objectifs du projet

- Permettre à un membre de la diaspora de retrouver d'autres membres autour de lui.
- Centraliser les annonces d'entraide (logement, cours, services, objets...).
- Centraliser les événements culturels, sociaux et de réseautage à venir.
- Fonctionner sans dépendance à une connexion internet permanente.

3. Public cible

Membres d'une communauté de diaspora installés à l'étranger, toutes générations, avec un accès internet parfois limité ou intermittent.

4. Fonctionnalités

4.1 Fonctionnalités essentielles (réalisées)

#	Fonctionnalité	Description
1	Authentification	Connexion par identifiant/mot de passe, création de compte, session conservée entre les lancements
2	Annuaire des membres	Liste des membres avec recherche (nom, prénom, pays, ville), fiche détail par membre
3	Annonces	Consultation, publication et suppression d'annonces (titre, contenu, catégorie, date)
4	Événements	Consultation, création et suppression d'événements (titre, date, lieu, description, catégorie)
5	Profil	Consultation et modification des informations personnelles, déconnexion

#	Fonctionnalité	Description
6	Navigation	Navigation principale par onglets entre les quatre pôles fonctionnels

4.2 Fonctionnalités hors périmètre (non retenues pour cette version)

- Messagerie privée entre membres
- Notifications push
- Synchronisation cloud multi-appareils

5. Contraintes techniques

- **Framework** : Flutter / Dart (SDK ^3.12.0)
- **Stockage** : local uniquement, base SQLite (sqflite) — choix motivé par un accès internet parfois limité pour le public cible, aucune synchronisation cloud n'est prévue
- **Session** : persistée via shared_preferences
- **Gestion d'état** : StatefulWidget / setState, accès aux données centralisé via un singleton (DbHelper)
- **Plateformes cibles** : Android (prioritaire, APK fourni), avec compatibilité iOS/macOS/web du fait de Flutter

6. Architecture technique

lib/	
main.dart	Point d'entrée, vérifie la session au démarrage
models/	Classes métier : Utilisateur, Annonce, Evenement
screens/	Écrans, un sous-dossier par fonctionnalité
widgets/	Widgets réutilisables (cartes membre/annonce/événement)
services/	DbHelper (accès SQLite), SessionService (session locale)
theme/	Thème visuel de l'application

7. Modèle de données

Utilisateur : id, nom, prénom, email, mot de passe, pays, ville, photo (optionnel), bio (optionnel)

Annonce : id, titre, contenu, auteurId, datePublication, catégorie

Événement : id, titre, date, lieu, description, organisateurId, catégorie

8. Écrans prévus (issus des wireframes)

1. Connexion / Inscription
2. Navigation principale (onglets)
3. Liste des membres + recherche
4. Détail d'un membre
5. Liste des annonces + ajout
6. Liste des événements + ajout
7. Profil (consultation + édition)

9. Critères de validation

- L'application se lance sans erreur bloquante

- Les quatre pôles fonctionnels (membres, annonces, événements, profil) sont accessibles et opérationnels
- Un parcours complet (connexion → consultation → ajout → suppression → déconnexion) fonctionne sans blocage
- flutter_analyze ne remonte aucune erreur
- Les tests unitaires et de widgets passent

10. Planning

	Phase	Contenu
Semaine 5		Cahier des charges, conception technique, choix d'architecture et de stockage
Semaine 6		Développement, tests, correction, build APK, dépôt GitHub